

SecureLock

How it works, explained from scratch

UNDER SEAL

A question paper leaks weeks before the paper exists.

This document explains what SecureLock does about that, assuming you know nothing about computers.

Written for exam board officials, controllers of examination, administrators, and anyone who has to understand this system well enough to explain it to somebody else. There is no prior technical knowledge assumed anywhere in these pages. Every technical word is explained the first time it appears, and again in the glossary at the end.

SecureLock is a working prototype built for the Smart India Hackathon. It runs entirely on one laptop, with no internet connection.

1. The problem, as it actually happens

When people hear that an examination paper has leaked, they usually picture the finished paper being stolen — a sealed envelope opened early, a PDF copied off a computer the night before the exam.

That does happen. But it is not where most of the risk lives.

A question paper is not born finished. It is assembled over weeks. A professor writes a question at home. She emails it to a colleague for a second opinion. He pastes it into a shared document. Someone prints a draft to read on a train. A coordinator collects thirty questions into one file and sends it around for approval.

By the time a paper exists as a single document worth guarding, its contents have already lived on a dozen laptops, in several inboxes, and in at least one printout that nobody can now account for.

The sentence worth remembering

Most systems protect the finished question paper. But the paper can be compromised long before it exists — while the individual questions are still being written and passed around.

So the useful question is not *how do we guard the final file?* It is *how do we account for every question, from the moment somebody types it, to the moment the exam begins?*

2. Why the usual protections are not enough

Three defences are normally offered. Each is reasonable, and each has a specific hole.

The usual protection	Where it falls short
Put the paper on a secure server with passwords	It protects the file once it exists, and does nothing for the weeks before. It also does nothing about the person who is <i>supposed</i> to have access.
Keep an access log of who opened what	A log is a file on the same server. Whoever administers that server can edit the log. A record that the record-keeper can quietly rewrite is not evidence.
Lock the paper with a timer until exam day	A timer has to read a clock. If it reads the server's clock, whoever runs the server can move it. If it reads the user's clock, any user can move it — changing your laptop's date is not a hack, it is a settings screen.

Notice that all three holes are the same shape. Each protection depends on somebody trustworthy — an administrator, a server, a clock under someone's control. The protection is only ever as good as the most privileged person in the room.

3. The idea, in one sentence

SecureLock secures the whole life of a question — not just the finished paper — and moves the two decisions that insiders can quietly corrupt, the record of what happened and the moment of release, somewhere no single administrator can reach.

Everything in the rest of this document is a consequence of that sentence.

4. Six ideas you need, in plain language

These are the only technical concepts in the system. Each one is ordinary; the contribution is the combination.

Encryption, and why every question gets its own lock

Encryption scrambles text so it is unreadable without a key. Think of a locked box: the box can sit on any shelf, in any office, and its contents stay private as long as the key is elsewhere.

SecureLock does something less obvious. It does not put the whole paper in one box. **Every single question gets its own box and its own key.** Steal one key and you have one question. There is no master moment where everything is readable at once.

The original, readable text of a question is never written to the database at all. Only the locked box is stored.

A fingerprint for text

A **hash** is a short code calculated from a piece of text — a fingerprint. SecureLock uses one called SHA-256, which always produces 64 characters, like this:

```
dc1e889d5def4ac244b4afc90dac11430ecd11ad3debbcad137d9fa7463cf371
```

Two properties make it useful. First, change anything at all in the question — one comma — and the fingerprint changes completely, so any edit is obvious. Second, it only works forwards: you can calculate the fingerprint from the question, but you cannot reconstruct the question from the fingerprint.

That second property is what makes it safe to publish. A fingerprint proves a question existed in an exact form at an exact time, while revealing nothing whatsoever about what the question says.

A record that cannot be quietly rewritten

Every action in SecureLock — a question written, read, refused, approved — is written to a log. The trick is in how the entries are joined: **each entry contains the fingerprint of the entry before it.**

This makes the log a chain. If somebody edits entry number 40, its fingerprint changes, so entry 41 no longer matches what it claims to follow, and neither does anything after it. You cannot alter one line quietly. You would have to rewrite every line that came after it — and the system checks the whole chain on demand.

The shared notebook (this is the blockchain part)

A blockchain is a record that many independent machines keep a copy of, where entries can only be added at the end and never edited. Because everyone holds a copy, one person changing their own copy changes nothing; it just makes their copy disagree with everyone else's.

The most common misunderstanding

The questions are **not** on the blockchain. No question text, no answers, no keys, and no personal data ever go anywhere near it. Only fingerprints, anonymous identifiers, timestamps, and the release condition. If the entire blockchain were published tomorrow, nobody would learn a single exam question from it.

So why use it at all? For exactly one reason: it is a place to put a record where the person who administers our database cannot reach it. That is the whole benefit, and it is a real one.

A rule that enforces itself

A **smart contract** is a small program stored in that shared notebook. Once it is there, it runs by itself, the same way for everybody, and nobody — including the people who built SecureLock — can talk it out of its rules.

SecureLock uses one to hold the paper: *do not permit this paper to be opened before this moment*. It is not a promise made by our software. It is a condition enforced somewhere else.

Whose clock decides

This is the subtle part, and it is worth getting right, because it is the part judges and officials ask about.

The smart contract does not read your laptop's clock, and it does not read our server's clock. It reads the **blockchain's own clock** — a timestamp agreed by the whole network as it records entries.

The practical consequence is simple and can be tested in front of you: set your computer's date forward to the year 2099 and ask for the paper. The answer is still no, and the refusal is recorded.

5. The life of one question paper

Here is the whole system in order. Nothing else happens; this is the complete journey.

Stage	What happens	What it leaves behind
1. Written	A setter types a question. It is locked with its own key before it is stored. The readable text is never saved.	A locked box, and a fingerprint.
2. Identified	The fingerprint is recorded in the shared notebook, along with an anonymous marker for who wrote it and when.	Proof this question existed in this exact form at this time.
3. Guarded	Another setter tries to open it and is refused. So is the administrator. Permission is checked on the server every time.	A recorded refusal — denials are evidence too.
4. Reviewed	A reviewer can read and approve, but can never edit. Once approved, the question is frozen — even against its author.	An approval, by a named role, at a known time.
5. Assembled	The system builds a paper from the approved pool, balancing topics and difficulty, avoiding near-duplicates, and spreading authorship across setters.	A paper nobody chose by hand — so no setter knows which of their questions was used, or against what.
6. Sealed	The finished paper is locked, its fingerprint recorded, and a release time attached to it in the smart contract.	A sealed paper and a public, checkable opening time.
7. Refused	Anyone asking for the paper early is refused — including the authority who sealed it, and including anyone who has changed their computer's clock.	More recorded refusals.
8. Opened	Once the blockchain's own clock passes the release time, the contract permits the release and the paper is unlocked.	A release nobody could bring forward.

6. Who is allowed to do what

Different people see genuinely different systems. This is enforced on the server for every single request — not by hiding buttons, which would be no protection at all.

Role	Can	Cannot
Question Setter	Write their own questions	See anyone else's question, or any assembled paper

Role	Can	Cannot
Reviewer	Read questions and approve them	Edit a question, ever
Exam Authority	Assemble, seal and release papers	Open a sealed paper early
Auditor	Verify every fingerprint and provenance record	Read the text of any question
Super Admin	Manage user accounts and inspect the audit trail	Read question content — the highest-privileged account in the system is deliberately not the most powerful one

That last row is the one worth pausing on. In most systems the administrator can see everything. Here, the administrator manages the system without being able to read what it protects.

7. Why blockchain here — and where it is not the answer

It would be easy, and wrong, to say “we used blockchain, therefore it is secure.” Blockchain is used in exactly two places, for reasons that can be stated plainly.

- **For the record of what happened.** A database administrator can rewrite database history. They cannot rewrite the shared notebook, so a tampered record no longer matches its anchor and the mismatch is detectable.
- **For the release condition.** A server clock can be changed by whoever runs the server. The blockchain's clock cannot be changed by that person, or by an end user on their own device.

Everything else — accounts, encrypted content, searching, the detailed audit metadata — lives in an ordinary database, because for those jobs an ordinary database is the correct tool and a blockchain would be a slower, worse one.

8. What SecureLock does not claim

Being straight about the limits is more persuasive than overclaiming, and an official who repeats an overclaim in public will regret it.

The claim we do not make	What is actually true
“Leak-proof”	Somebody authorised to read a question can photograph the screen. No software prevents that. The goal is to reduce exposure and make access traceable.
“It proves who read a question”	It proves an authorised account requested it through this system, at a recorded moment. A person and an account are not the same thing.
“The blockchain clock is perfectly accurate”	It has some tolerance. The honest claim is narrower and still strong: an end user cannot move it from their own device.
“AI secures the paper”	The assembly engine makes a paper less predictable. That is a change in the odds, not a guarantee — and it follows fixed rules rather than being a language model.
“It works even if the blockchain is down”	If the network is unreachable, release is refused rather than falling back to a server clock. That is the safe direction to fail, and it is a real operational constraint.

What is claimed: tamper-evident records, defence in depth, traceable provenance, a blockchain-enforced release, and insider risk that is reduced and — when something does happen — investigable.

9. Explaining it in sixty seconds

If you have one minute and a non-technical audience, this works:

The sixty-second version

“Exam papers don't usually leak from the sealed envelope. They leak weeks earlier, while the questions are still being written and emailed around. SecureLock locks every question separately the moment it's typed — each with its own key, so there's never a moment when everything is readable at once.

Every hand-off is recorded in a log where each entry is sealed to the one before it, so a single line can't be quietly edited. And the finished paper is held shut by a rule stored on a blockchain, which decides the opening moment using its own clock — not our server's, and not yours. You can change your computer's date to 2099 in front of me, ask for the paper, and it will still say no.

We're not claiming it's leak-proof — anyone who can see a question can photograph it. We're claiming that every access is traceable, tampering is visible, and no single administrator can open the paper early.”

10. Glossary

Word	Meaning
Encryption	Scrambling text so it is unreadable without the key.
AES-256-GCM	The specific, standard method used to do that.
Key	The secret that unlocks one encrypted item.
Hash / SHA-256	A 64-character fingerprint of a piece of text. Changes completely if the text changes; cannot be reversed.
Blockchain	A shared, append-only record kept by many machines, so no single one of them can rewrite history.
Smart contract	A small self-enforcing program stored on the blockchain.
Time lock	The rule that refuses to release the paper before a set moment.
block.timestamp	The blockchain's own clock — the one the time lock reads.
Provenance	The recorded history of where something came from and who handled it.
Audit trail	The chained log of everything that happened, including refusals.
Hash chain	A log where each entry carries the fingerprint of the previous one, so edits break the chain.